

libc なしで動かす — システムコールを直接呼ぶ

今日のゴール

`printf` も `exit` も使わずに、システムコールを直接発行して「文字を表示して終了コードを返す」プログラムを動かす。さらに、自作コンパイラが出力した C プログラムを **libc** なしで実行する。

1 この回の位置づけ

- ・ランタイム発展シリーズの第 1 回（選択制）。前提はコマ 11 (`printf`) まで
- ・コマ 1 の資料で「アセンブルとリンクは `gcc` に、実行は `qemu` に任せる」と言った、その任せていた部分を 1 つ開ける
- ・続く R2（自前 `printf`）・R3（自前 `malloc`）の土台になる

2 いま何が起きているか

`printf("hi\n")` を呼ぶと、最終的に何が起きているのか。

<code>printf("hi\n")</code>	← あなたのプログラム
→ <code>libc</code> の <code>printf</code>	← 書式を解釈し、バッファに詰める
→ <code>libc</code> の <code>write</code>	← システムコールの薄い包み紙
→ <code>ecall</code> 命令	← ここで初めて OS に頼む
→ Linux カーネル	← 実際に画面へ出す

いままで `riscv64-linux-gnu-gcc -static` でリンクすると、`libc`（C 標準ライブラリ）が自動的にくっついていて、`libc` は `printf` だけでなく、プログラムの起点 `_start` も提供している。

`_start (libc)` → 環境変数などの準備 → `main` を呼ぶ → `main` の戻り値で `exit`

`main` が「プログラムの入口」に見えていたのは、`libc` がそう見せていたからである。

3 システムコール — OS への頼み方

画面に文字を出す、ファイルを読む、プログラムを終える。これらは自分のプログラムだけでは実現できず、OS に頼む必要がある。その頼み方がシステムコールである。

RV64 Linux では、次の約束になっている。

決めごと	場所
どのシステムコールか（番号）	<code>a7</code>
第 1 引数、第 2 引数、第 3 引数…	<code>a0, a1, a2, …</code>
発行する命令	<code>ecall</code>
戻り値	<code>a0</code>

引数の渡し方が関数呼び出しとほぼ同じなのがポイントである（違うのは、番号を `a7` に入れて `call` ではなく `ecall` を使うこと）。

この回で使うのは 2 つだけである。

番号	名前	引数	意味
64	<code>write</code>	<code>fd, buf, count</code>	<code>fd</code> へ <code>buf</code> から <code>count</code> バイト書く
93	<code>exit</code>	<code>code</code>	プログラムを終了する

`fd`（ファイル記述子）は、`1` が標準出力である。

4 Step 1: hello.s — 自分で `_start` を書く

`libc` を外すと `_start` がなくなるので、自分で用意する。

```
.data
msg:
.ascii "Hello, no libc!\n"
.set msglen, 16
```

```
.text
.globl _start
_start:
    # write(1, msg, msglen) をここに書く
    # exit(42) をここに書く
```

`.ascii` は、コマ 11 の `.byte` 列と同じことを文字列で書ける指示である（NUL 終端は付かない。`write` はバイト数を渡すので終端が要らない）。

重要

`_start` の最後は `ret` ではない。`ret` は「呼び出し元へ戻る」命令だが、`_start` には呼び出し元がない。必ず `exit` システムコールで終わること。忘れると、`_start` の直後にあるゴミを命令として実行して異常終了する。

ビルドと実行は次の通り。`-nostdlib` が「libc を付けない」指示である。

```
riscv64-linux-gnu-gcc -nostdlib -static hello.s -o hello
gemu-riscv64 ./hello; echo $?
```

5 Step 2: 自作コンパイラの出力を `libc` なしで動かす

次に、自分のコンパイラが出力した C プログラムを `libc` なしで動かす。必要なのは 2 つの橋渡しである。

1. C から呼べる `sys_write` / `sys_exit` (`ecall` は `mycc` が生成できないため)
2. `main` を呼ぶ `_start`

これらをまとめた `syscall.s` を配布する（完成品）。中身は次の形である。

```
.globl sys_write
sys_write:
    li    a7, 64          # 引数 a0, a1, a2 はそのまま使える
    ecall
    ret

.globl _start
_start:
```

```
call main
li a7, 93      # exit(main の戻り値)
ecall
```

`sys_write` が短いことに注目してほしい。C の呼び出し規約 (`a0, a1, a2`) とシステムコールの規約 (`a0, a1, a2`) が同じなので、番号を入れて `ecall` するだけでよい。libc の `write` も、実体はこれとほとんど同じである。

C 側では、外部関数として宣言して普通に呼ぶ。

```
int sys_write(int fd, char *buf, int len);

int main() {
    sys_write(1, "hi from C\n", 10);
    return 7;
}
```

```
python3 ../../../../final/mycc.py tests/print_loop.c > prog.s
riscv64-linux-gnu-gcc -nostdlib -static prog.s syscall.s -o prog
qemu-riscv64 ./prog; echo $?
```

自作コンパイラの出力が、libc なしで OS と直接やり取りして動いた。このプログラムに含まれる命令は、すべて自分のコンパイラか、自分が書いたアセンブリから来ている。

6 テスト

```
python3 check.py
```

Step 1 (`hello.s` の表示と終了コード) と Step 2 (`tests/*.c` の実行) を続けて確認する。

7 発展課題

1. 文字列の長さを数える: `sys_write` に渡す長さを固定値ではなく、NUL 終端を数えて求める `my_strlen` を Core C で書く (R2 の下準備)
2. 標準エラー出力: fd を 2 にして書き、`qemu ... 2>/dev/null` で消えることを確か

める

3. 他のシステムコール: `read` (番号 63) で標準入力から 1 文字読むプログラムを書く
4. `libc` のサイズ: `-static` あり／なしで実行ファイルのサイズを比べる (`ls -l`)。libc がどれだけ大きいかわかる
5. `_start` の引数: 本物の `_start` は、スタックから `argc / argv` を取り出して `main(argc, argv)` に渡している。sp の指す先を `gdb-multiarch` で覗いてみる

補足

コラム: なぜ `ecall` が必要なのか。 ユーザーのプログラムは、画面やファイルに直接触れない（触れたら他のプログラムを壊してしまう）。`ecall` は「特権レベルを上げてカーネルに制御を渡す」専用の命令で、これが唯一の入口になっている。x86-64 では `syscall` 命令、番号は `rax` に入れるなど、名前と場所は違うが仕組みは同じである。

8 次回予告

`write` が使えるようになった。R2 では、この上に **自前の `printf`** を作る。整数を 10 進の文字列に変換する処理を、Core プロファイルの C で書く。